Smartband V001 — Rev1 Firmware

CCS 21 Migration & Debug Session — Detailed Technical Report

Prepared for Prof. Giovanni · Masrur Jamil Prochchhod · Hochschule Hamm-Lippstadt

Project: Smartband V001 · Board: Rev1 (CC2640R2FRSMR) · SDK: SimpleLink CC2640R2 5.30.1.11

---

1. Purpose and Scope

Rev1 firmware development for Smartband V001 had been carried out entirely under Code Composer Studio (CCS) 12.8.1, using the TI BLE5-Stack architecture (a two-project split between a compiled BLE stack library and the application). Partway through the project, CCS 12.8.1 began failing to enumerate the onboard XDS110 debug probe with a Windows "Code 28" driver error that could not be resolved within the existing install.

This forced a migration to CCS 21, a substantially newer IDE built on a different underlying framework (Eclipse-based CCS 12.8.1 vs. the Theia-based CCS 21 UI), paired with a materially newer default toolchain (TI ARM Clang instead of the classic ti-cgt-arm compiler this SDK generation expects).

This report documents, in technical detail, every issue encountered during that migration and how each was diagnosed and resolved: the toolchain setup, ten independent build-system failures, the hardware debug-access lockout that followed, firmware validation, and the standalone-power issue that remains open. Source excerpts, exact error text, and the diagnostic commands used are included throughout so the reasoning is reproducible, not just the outcome.

2. Executive Summary

Area	Status	Notes
Toolchain install (CCS 21 + classic compiler)	**DONE**	Manual ti-cgt-arm 20.2.7.LTS install required
stack_library project build	**RESOLVED**	Clean build; see §4
app project build	**RESOLVED**	Clean build; see §5
XDS110 debug/flash access	**RESOLVED**	CCFG debug lock; recovered via UniFlash; see §6
BLE advertising / GATT	**CONFIRMED**	"Simple Peripheral" visible & connectable
OLED + I2C sensors (debug-attached)	**WORKING**	Confirmed while LaunchPad attached; see §7
Standalone cold-boot power	**OPEN**	Power lost entirely on PeakTech cycle; see §8

3. Toolchain Migration

3.1 Trigger

CCS 12.8.1's XDS110 driver failed with Windows Device Manager error Code 28 ("The drivers for this device are not installed"), and reinstalling the driver package within CCS 12.8.1 did not resolve it. TI's own CCS 12.8.1 release notes describe a fix for a similar class of XDS110 driver installation issue, but it did not apply cleanly on this machine (likely due to the Windows build in use, 10.0.26200.x).

3.2 Components installed

- CCS 21 — "Wireless Connectivity MCU" workload only (not the full install)
- ti-cgt-arm 20.2.7.LTS — the classic ARM compiler, installed manually and separately, because CCS 21 defaults to TI ARM Clang (ti-cgt-armllvm), which this SDK generation's BLE5-Stack source is not written against
- SimpleLink CC2640R2 SDK 5.30.1.11 — reused unchanged from the CCS 12.8.1 setup
- XDCtools 3.51.3.28_core — reused unchanged

3.3 Key difference from CCS 12.8.1

CCS 12.8.1's project importer/generator (Eclipse CDT-based) produced a fully correct, working project the first time, with all include paths, predefined symbols and linker settings populated automatically from the SDK's .projectspec template. CCS 21's newer project handling (a mixed Eclipse-CDT-backend / Theia-frontend architecture) did not reproduce this reliably when the existing project was re-imported — each of the following ten sections documents one place where the reproduction diverged from a working configuration.

4. Resolving the stack_library Build

The ble5_simple_peripheral_cc2640r2lp_stack_library project compiles the BLE5-Stack into a static .lib consumed by the app project. Getting this project to build clean under CCS 21 required five separate fixes, found and verified one at a time by rebuilding after each change and reading the resulting compiler/linker output.

4.1 Wrong build configuration was being edited

This project has three build configurations — FlashROM_Library (active), FlashROM_Library_PTM, and (in the app project) FlashROM_StackLibrary_RCOSC. Every GUI change made through Project Properties appeared to save successfully, but repeated rebuilds showed the same missing include paths and defines. Rather than continue guessing through the GUI, the actual .cproject XML was inspected directly:

Diagnostic command used:

```
findstr /n "FlashROM_Library\" FlashROM_Library_PTM\"
  COM_TI_XDC_INCLUDE_PATH npi/src" .cproject
```

This confirmed that every include path added through the GUI had been landing inside the FlashROM_Library_PTM configuration block (starting at a later line in the XML) rather than the active FlashROM_Library block (lines 5–101). Once corrected — adding paths explicitly under the FlashROM_Library configuration and re-verifying with the same findstr check before every rebuild — the paths took effect.

4.2 Missing workspace-level SDK path variable

The build initially failed with "69 unresolved linked resources." This was traced to the path variable COM_TI_SIMPLELINK_CC2640R2_SDK_INSTALL_DIR being unset at the workspace level. It had to be defined via CCS → Window/IDE Settings → Path Variables (workspace scope), not at the per-project Variables

page and not as a Windows environment variable — CCS's resource-linking system specifically reads the workspace-level entry.

4.3 Missing XDCtools packages include path

With the correct configuration now being edited, several files still failed with:

```
fatal error #1965: cannot open source file "xdc/std.h"
```

This is supplied by XDCtools, not the SDK. The fix was adding the literal path to the active configuration's include list:

```
C:/ti/xdctools_3_51_03_28_core/packages
```

Note: CCS 21's Add Include Path dialog rejected the variable form ${COM_TI_XDC_INCLUDE_PATH} with a "resolves to a non-existent location" warning, even though that same variable form works correctly elsewhere in the SDK's own generated project templates. The literal, resolved path was used instead to work around this GUI validation quirk.

4.4 Missing / duplicated NPI include path

A second missing header appeared:

```
fatal error #1965: cannot open source file "inc/npi_ble.h"
```

A directory search revealed two copies of npi_ble.h in the SDK, under source/ti/ble5stack/npi/src/inc and source/ti/blestack/npi/src/inc (two parallel, differently-cased SDK trees ship side by side in 5.30.1.11). The correct one for this project (matching every other ble5stack path already in use) was:

```
C:/ti/simplelink_cc2640r2_sdk_5_30_01_11/source/ti/ble5stack/npi/src/inc
```

Note the SDK source itself expects the parent of inc/, i.e. .../npi/src (without the trailing /inc) — the include statement in npi.c is #include "inc/npi_ble.h", a relative path assuming the search root is one level up.

4.5 Files that must be excluded from the build

Two files failed with an explicit, intentional compiler directive:

```
fatal error #35: #error directive: Callback Timer module
  shouldn't be included (no callback timer is needed)!
```

This affected osal_cbtimer.c and rom_init.c. These are guarded by an #error specifically to prevent them from being compiled in this build configuration; they were excluded from the build via right-click → Exclude from Build on the active configuration (confirmed by them disappearing from the compile-file list in the next build log).

4.6 C89 vs. C99 dialect

osal.c failed to compile:

```
osal.c, line 1881: error #256: type name is not allowed
osal.c, line 1881: error #66: expected a ";"
```

osal.c — offending line:

```
for (uint8 i = 0; i < tasksCnt; i++)
```

This is a C89-vs-C99 issue: the loop counter is declared inline inside the for(), which C89 (the compiler's apparent default dialect in this configuration) does not permit. Enabling --c99 on the active configuration's Arm Compiler settings resolved this without touching the SDK source.

4.7 Stale macro name in vendor source

mb_patch.c failed with:

```
error #20: identifier "INT_RFC_CPE1" is undefined
error #20: identifier "INT_RFC_CPE0" is undefined
```

A search of the SDK's own driverlib header showed the real, current macro names:

source/ti/devices/cc26x0r2/inc/hw_ints.h:

```
#define INT_RFC_CPE_1     18  // Combined Interrupt for CPE
#define INT_RFC_CPE_0     25  // Combined Interrupt for CPE
```

This SDK's BLE5-Stack mb_patch.c had not been updated in step with its own driverlib headers in this release — the macro gained an underscore before the trailing digit at some point. mb_patch.c was patched directly to use the underscored form (INT_RFC_CPE_1 / INT_RFC_CPE_0).

4.8 USE_ICALL / ICALL_LITE define

Once compiling, osal.c and osal_icall_ble.c both failed with a cluster of undefined ICall identifiers (ICall_EntityID, OSAL_EVENT_SERVICE, osal_service_entry, and others). Their common ancestor in osal.c is:

osal.c:

```
#ifdef USE_ICALL
#ifdef ICALL_JT
  #include "icall_jt.h"
#else
  #include <icall.h>
#endif /* ICALL_JT */
#endif /* USE_ICALL */
```

The active configuration defined ICALL_JT but not the outer guard, USE_ICALL, so the entire block — including the include of icall_jt.h — was skipped by the preprocessor. Adding USE_ICALL as a bare predefined symbol resolved every identifier in this cluster in one pass.

4.9 Result

With all of the above applied, ble5_simple_peripheral_cc2640r2lp_stack_library built cleanly, producing ble5_simple_peripheral_cc2640r2lp_stack_library.lib.

5. Resolving the app Project Build

With the stack library building, the ble5_simple_peripheral_cc2640r2lp_app project surfaced a further, largely independent set of issues — several of which turned out to be more consequential than they first appeared.

5.1 Same USE_ICALL fix, different project

As with the stack library, USE_ICALL had to be added separately to the app project's own predefined symbols — each CCS project's build settings are entirely independent, so a fix applied to one project has no effect on the other.

5.2 The STACK_LIBRARY define — root cause of a cascading failure

After adding USE_ICALL, two new, seemingly unrelated errors appeared:

```
error #20: identifier "IDX_Gap_RegisterConnEventCb" is undefined
error #20: identifier "MAX_NUM_BLE_CONNS" is undefined
```

The first instinct was to add ICALL_LITE (which the missing identifiers' surrounding code is guarded by). That define did fix icall_user_config.c, but reintroduced the same two errors from a different file — because icall_api_idx.h branches on a second, independent condition:

icall/inc/icall_api_idx.h:

```
#ifndef STACK_LIBRARY
  #include "ble_dispatch_lite_idx.h"
#else
  // ... real, populated #define IDX_... table ...
#endif
```

Without STACK_LIBRARY defined, the app pulled in ble_dispatch_lite_idx.h — which, on inspection, is a code-generation template file, still containing literal // <<INSERT:...>> placeholder markers rather than a complete, real index table. It does not define IDX_Gap_RegisterConnEventCb at all, which is exactly the symbol the app's own SimplePeripheral_startAutoPhyChange() calls.

The actual, correct fix was located by comparing against TI's original project template (.projectspec) for this exact board/example, which lists the complete, correct predefined-symbol set for this build configuration:

TI reference .opt file (excerpt):

```
-DBOARD_DISPLAY_USE_LCD=0
-DBOARD_DISPLAY_USE_UART=1
-DBOARD_DISPLAY_USE_UART_ANSI=1
-DCC2640R2_LAUNCHXL
-DCC26XX
-DCC26XX_R2
-DDeviceFamily_CC26X0R2
-DICALL_EVENTS
-DICALL_JT
-DICALL_LITE
-DICALL_MAX_NUM_ENTITIES=6
-DICALL_MAX_NUM_TASKS=3
-DICALL_STACK0_ADDR
-DMAX_NUM_BLE_CONNS=1
```

```
-DPOWER_SAVING
-DRF_SINGLEMODE
-DSTACK_LIBRARY
-DTBM_ACTIVE_ITEMS_ONLY
-DUSE_ICALL
-Dxdc_runtime_Assert_DISABLE_ALL
-Dxdc_runtime_Log_DISABLE_ALL
```

The active CCS 21 configuration's define list was missing STACK_LIBRARY, CC26XX_R2, ICALL_MAX_NUM_ENTITIES, ICALL_MAX_NUM_TASKS, MAX_NUM_BLE_CONNS, POWER_SAVING, RF_SINGLEMODE, TBM_ACTIVE_ITEMS_ONLY, BOARD_DISPLAY_USE_*, and the two xdc_runtime_*_DISABLE_ALL symbols, and additionally carried two symbols that do not belong in this project at all (BLE_V50_FEATURES and CC2650EM_7ID — the source of a recurring “incompatible redefinition of macro CC2650EM_7ID” warning seen in every build log up to this point). The full symbol list was replaced wholesale to match the reference set.

With STACK_LIBRARY correctly defined, icall_api_idx.h takes its populated branch, and both previously-undefined identifiers resolved without any further source changes.

5.3 Linker library list

Once every file compiled, the link stage failed with dozens of unresolved symbols (GAP_*, GATT_*, PIN_*, UART_*, RF_*, Display_*, and others), plus a memory-map error. The active configuration's linker Library list had, at some point, been reduced to just libc.a. The full list, matching the working _PTM/_RCOSC sibling configurations, was restored:

```
driverlib.lib
dpl_cc26x0r2.aem3
drivers_cc26x0r2.aem3
rf_singleMode_cc26x0r2.aem3
display.aem3
grlib.a
ble_r2.symbols        (from FlashROM_Library)
lib_linker.cmd        (from FlashROM_Library)
ble5_..._stack_library.lib   (from FlashROM_Library)
cc26xx_app.cmd        (BLE memory map / linker script)
libc.a
```

cc26xx_app.cmd in particular is essential: it supplies the MEMORY{} section definitions and the SNV/ICall boundary sections the BLE stack depends on, which is why omitting it produced “program will not fit into available memory” and “DEFAULT memory range overlaps” errors even once every symbol was otherwise resolved.

5.4 Duplicate outer/inner project folders

A structural issue, separate from any compiler/linker setting, repeatedly disrupted CCS 21's Theia-based project importer: the exported project directory contained a folder named identically to its own parent (…\ble5_simple_peripheral_cc2640r2lp_app\ble5_simple_peripheral_cc2640r2lp_app\), an artifact of how the original .projectspec-based export laid the project out.

CCS 21's Import CCS Projects dialog was inconsistent about which of the two levels it treated as the actual project root, at times importing the outer wrapper as a plain, unbuildable folder, and at least once importing an entirely unrelated TI SDK example project it found nested nearby. This was resolved structurally: both projects (app and stack_library) were moved out from under the duplicate-named wrapper folder directly into C:\ti\smart_band_v001_app\, so each has exactly one, unambiguous project directory.

5.5 Result

With the corrected symbol list, restored linker library list, and flattened folder structure, ble5_simple_peripheral_cc2640r2lp_app built and linked cleanly, producing ble5_simple_peripheral_cc2640r2lp_app.out and the corresponding .hex.

6. XDS110 Debug Access Recovery

With a clean build in hand, flashing failed — not due to any remaining build issue, but due to a hardware-level debug-access problem on the CC2640R2F itself. This was isolated methodically by working through each distinct error code returned by the XDS110/IcePick_C connection sequence, in the order encountered.

6.1 Error -242 — router subpath inaccessible

```
Error connecting to the target: (Error -242 @ 0x0)
A router subpath could not be accessed. The board
configuration file is probably incorrect.
```

Investigated and ruled out, in order: P10 jumper position (XDS110 Power vs. Extern Pwr), physical JTAG wiring on the loose-wired P1 header (TCK/TMS specifically, since these are required for the IcePick router to respond at all), and the XDS110 probe's own firmware/driver, confirmed current (3.0.0.43) via CCS's bundled xdsdfu.exe utility and clean in Windows Device Manager with no warning icons on either “XDS110 Class Data Port” or “XDS110 Class Debug Probe.”

6.2 Error -2131 and -241 — the real cause

```
IcePick_C: Error connecting to the target:
(Error -241 @ 0x0) A router subpath could not be
accessed. A security error has probably occurred.
Make sure your device is unlocked.
```

SC_ERR_ROUTER_SECURE_SUBPATH is a documented CC2640R2 failure mode: the CCFG (Customer Configuration) region's debug-access bits can end up in a locked state, which blocks JTAG at a security level rather than an electrical one. This is unrelated to wiring and would explain every prior symptom (including the earlier -242, if the router itself refuses full access while locked).

6.3 Recovery via UniFlash forced mass erase

TI UniFlash was used (Program → Settings & Utilities → Manual Erase → “Erase entire flash” → Erase Entire Flash) to perform a forced mass erase, which is the standard, TI-documented recovery path for a CC2640R2 with a locked debug interface. Note (documented for completeness, not because it applies here): if a device's mass-erase-disable lock bit has itself been set, no tool can recover it — this is a separate, rare, deliberately-configured option and was not the case on this board.

After the forced mass erase completed, CCS 21 was able to connect, flash, and debug the board normally.

7. Firmware Validation

7.1 BLE

The board advertises as “Simple Peripheral” and is visible and connectable from nRF Connect on a mobile device, with the expected GATT services (Device Information, GAP GATT Service, and the custom Simple GATT Profile) present.

7.2 OLED and I2C sensors

The SSD1306 OLED and the three I2C-bus sensors (LIS331 accelerometer, MAX30102 heart-rate/SpO2, MAX30205 temperature) were all confirmed functional while the LaunchPad/XDS110 remained attached and providing power via XDS110 Power mode.

7.3 Cold-boot startup delay (attempted mitigation)

Initial standalone testing showed the OLED producing garbled output on some power-ups. As a first hypothesis, a startup delay was added before I2C/OLED initialisation in SimplePeripheral_init(), to rule out a race between power-rail settling and the first I2C bus traffic:

Application/simple_peripheral.c — SimplePeripheral_init():

```
dispHandle = Display_open(SBP_DISPLAY_TYPE, NULL);

/* Give the power rail, crystal, and I2C peripherals time to
 * settle on a genuine cold boot (no debugger attached). */
Task_sleep(200000 / Clock_tickPeriod);   // ~200ms settle time

/* --- I2C SENSOR TEST --- */
if (dispHandle != NULL) {
    I2C_init();
    uint8_t lis_whoami = get_i2c_value_lis331_whoami();
    ...
```

Tested at both 200 ms and 500 ms; neither resolved the issue. This was superseded once the true, larger-scope symptom was identified (§8): the board was losing power entirely, not merely producing corrupted I2C data on an otherwise-successful boot. This delay remains in the source as a reasonable, low-cost startup safeguard, but it does not address the open issue below.

8. Open Issue: Standalone Cold-Boot Power Loss

With the LaunchPad fully disconnected and the board powered only from a PeakTech bench supply (P10 jumper set to Extern Pwr), cycling the PeakTech's output off and back on causes the board to lose power entirely rather than reboot — no OLED activity, no LEDs, nothing on a subsequent power-up. This is a distinct and more fundamental symptom than the earlier OLED-garbling behaviour, and points to the power path itself, upstream of anything firmware can influence.

8.1 Passive-component verification

Because the initial symptom (garbled OLED) looked I2C-related, the I2C bus and reset circuit passives were checked first, using a multimeter in resistance mode with the board fully unpowered:

Check	Expected	Measured
R103 — I2C SCL pull-up to 3V3	~10 kΩ	~10 kΩ — OK
R104 — I2C SDA pull-up to 3V3	~10 kΩ	9.92 kΩ — OK
R1 — nRESET pull-up to 3V3	~100 kΩ	98.8 kΩ — OK
R17 / R18 — JTAG_TDI / TDO pulls (schematic: DNM)	Open (not populated)	Open — OK, as designed

8.2 Firmware-side I2C configuration verification

The I2C0 pin mapping used by the board support file was also inspected directly, since the schematic's own annotation ("Not needed. Perhaps internal pullups on CC2640?" next to R103/R104) raised the possibility that firmware might be relying on internal pull-ups on the wrong pins:

boards/CC2640R2_LAUNCHXL/CC2640R2_LAUNCHXL.c:

```
.sdaPin = IOID_6,
.sclPin = IOID_5,
// ...
// NOT using CC2640R2_LAUNCHXL_I2C0_SCL0/SDA0 -- wrong for Rev1.
IOID_5 | PIN_INPUT_EN | PIN_PULLUP,   /* SCL */
IOID_6 | PIN_INPUT_EN | PIN_PULLUP,   /* SDA */
```

This confirmed the pin mapping had already been corrected for Rev1 in an earlier session (the header file's default I2C0_SCL0/SDA0 macros point at IOID_4/IOID_5, which is wrong for this board; the .c file overrides this correctly to IOID_5/IOID_6, matching the schematic, with internal pull-ups also explicitly enabled as a second line of defence alongside the physical R103/R104).

8.3 Conclusion of this session

Every electrically-relevant passive component checked measures correctly, and the I2C0 pin mapping and pull-up configuration in firmware are both correct. Since the actual fault — complete loss of power on a PeakTech-only cold cycle — is not explained by any of the above, the fault most likely sits in the VBAT power path itself: the P10 jumper header, or the trace/solder joint between P10's Extern Pwr position and P1 pin 16 (VBAT). This is consistent with power working correctly whenever the LaunchPad is attached (XDS110 Power mode routes VBAT through a completely different physical path than Extern Pwr mode does).

8.4 Suggested next steps

- Continuity check (board fully unpowered, multimeter in resistance/continuity mode) from the Extern Pwr side of the P10 header through to P1 pin 16 (VBAT)
- Close visual inspection of the P10 header itself for a cold solder joint, lifted pad, or bent pin
- Once the LiPo battery and TP4056 charge module arrive, repeat the standalone cold-boot test powered from battery rather than PeakTech — this will determine whether the PeakTech's own current-limiting behaviour is a contributing factor, independent of the board fault itself

9. Key Learnings Carried Forward

- When a CCS project silently behaves as if a change was never applied, trust the raw .cproject XML (findstr /n on the exact setting) over the GUI — the GUI can visually confirm a save while writing to the wrong build configuration, with no error shown.
- Generated build files — ccsIncludes.opt, the makefile itself, compiler.opt — are regenerated by CCS from .cproject on every build and must never be hand-edited directly; a hand-edit will silently vanish on the next build.
- USE_ICALL, ICALL_LITE and STACK_LIBRARY must be defined together, consistently, in both the app and stack_library projects. Missing STACK_LIBRARY specifically and silently redirects the ICall API index layer onto an incomplete, template-stage source file rather than the real, generated index — producing "undefined identifier" errors that look like a missing include, not a missing #define.
- A CC2640R2F reporting a router or security-subpath error (-241/-2131) during a debug connect attempt is very likely a CCFG debug-interface lock, not a wiring fault — recoverable via a forced mass erase in TI UniFlash, and not something further wiring changes will fix.
- Duplicate-named outer/inner project folders, an artifact of TI's .projectspec-based export process, are a recurring and easy-to-miss trap for CCS 21's Theia-based project importer. Keep one, unambiguous project directory per project rather than the nested export structure.
- When a board's schematic carries an uncertain designer note (e.g. "Not needed. Perhaps internal pullups?") about a component, verify electrically rather than assuming either answer — in this case the resistors were in fact populated and correctly valued, ruling out what looked like the most likely explanation for the original symptom.